

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Д32: Проектирование и технический дизайн API

Выполнил:

Зеленков Евгений

**Группа
К3342**

**Проверил:
Добряков Д. И.**

Санкт-Петербург

2026 г.

Задача

1. Описать функциональные и нефункциональные требования к backend-приложению.
2. Выбрать формат реализации API: REST API или Backend For Frontend.
3. Спроектировать эндпоинты API в формате OpenAPI-схемы с учетом диаграммы БД из ДЗ1.
4. Описать тела запросов, тела ответов и возможные ошибки API.
5. Подготовить Swagger-документацию для обсуждения методов на защите.

Ход работы

В рамках работы был спроектирован REST API для приложения бронирования столиков в ресторанах. API основан на диаграмме базы данных из ДЗ1 и покрывает основные сущности: пользователей, рестораны, кухни ресторанов, столики, меню, позиции меню, бронирования, отзывы и фотографии.

Для реализации выбран REST API. Такой подход подходит для данной предметной области, так как модель данных состоит из явно выделенных ресурсов, а операции над ними удобно выражаются стандартными HTTP-методами GET, POST, PATCH и DELETE. Backend For Frontend не использовался, так как в рамках текущей работы не требуется проектировать разные API-контракты для нескольких типов клиентских приложений.

В OpenAPI-схеме используется префикс /api/v1. Авторизация описана через Bearer JWT: после входа пользователь получает токен и передает его в заголовке Authorization. Публичные методы просмотра ресторанов, меню и отзывов доступны без токена, а создание бронирований, отзывов и административные операции требуют авторизации.

Функциональные требования

- Пользователь может зарегистрироваться и войти в систему по номеру телефона и паролю.
- Пользователь может получить и обновить данные своего профиля.
- Пользователь может просматривать список ресторанов и фильтровать его по кухне, рейтингу, среднему чеку и строке поиска.
- Пользователь может открыть карточку ресторана, посмотреть кухни, меню, позиции меню, столики и отзывы.
- Авторизованный пользователь может создать бронь на конкретный ресторан, столик и время.
- Авторизованный пользователь может просматривать свои бронирования и отменять активную бронь.
- Авторизованный пользователь может оставить отзыв по завершенной брони и прикрепить к нему фотографии.
- Администратор может управлять ресторанами, кухнями, столиками, меню, позициями меню, статусами бронирований и модерацией отзывов.

Нефункциональные требования

- API использует JSON для тел запросов и ответов.
- В промышленной среде API должен работать по HTTPS.
- Пароль пользователя не возвращается в ответах API и должен храниться в базе данных в виде хеша.
- Ошибки API возвращаются в едином формате с кодом ошибки и текстовым сообщением.
- Для списочных методов предусмотрена пагинация через параметры page и limit.
- API должен возвращать корректные HTTP-статусы: 200, 201, 204, 400, 401, 403, 404, 409, 422 и 500.
- Пользователь не должен иметь доступ к чужим бронированиям и административным операциям.
- Неопубликованные меню и блюда не должны отображаться обычным пользователям.

Основные эндпоинты

Метод	Эндпоинт	Назначение
POST	/auth/register	Регистрация пользователя
POST	/auth/login	Авторизация и получение JWT-токена
GET	/users/me	Получение профиля текущего пользователя
PATCH	/users/me	Обновление профиля текущего пользователя
GET	/restaurants	Список ресторанов с фильтрацией и пагинацией
POST	/restaurants	Создание ресторана администратором
GET	/restaurants/{restaurantId}	Получение карточки ресторана
PATCH	/restaurants/{restaurantId}	Обновление ресторана администратором
GET	/restaurants/{restaurantId}/cuisines	Получение кухни ресторана
POST	/restaurants/{restaurantId}/cuisines	Добавление кухни ресторану
GET	/restaurants/{restaurantId}/tables	Получение столиков ресторана
POST	/restaurants/{restaurantId}/tables	Создание столика ресторана
GET	/restaurants/{restaurantId}/menus	Получение меню ресторана
POST	/restaurants/{restaurantId}/menus	Создание меню ресторана
GET	/menus/{menuId}/items	Получение позиций меню
POST	/menus/{menuId}/items	Создание позиции меню
POST	/reservations	Создание бронирования
GET	/reservations	Получение бронирований текущего пользователя
PATCH	/reservations/{reservationId}/cancel	Отмена бронирования
PATCH	/reservations/{reservationId}/status	Изменение статуса бронирования
GET	/restaurants/{restaurantId}/reviews	Получение отзывов ресторана
POST	/reviews	Создание отзыва по завершенной брони
PATCH	/reviews/{reviewId}/verification	Модерация отзыва администратором

Тела запросов и ответов

В OpenAPI-схеме описаны DTO для всех основных операций. Например, RegisterRequest содержит имя, дату рождения, телефон и пароль; LoginRequest содержит телефон и пароль; CreateReservationRequest содержит restaurantId, tableId, reservedAt, guestCount и необязательный комментарий; CreateReviewRequest содержит reservationId, rating и комментарий.

Ответы API возвращают доменные объекты без технически чувствительных данных. Например, User не содержит пароль, RestaurantDetails содержит данные ресторана и список кухонь, Reservation содержит пользователя, ресторан, столик, время бронирования и статус, а Review содержит ссылку на бронирование, оценку, комментарий и фотографии.

Ошибки API

Код	Описание
400	Некорректный запрос
401	Пользователь не авторизован
403	Недостаточно прав для выполнения операции
404	Запрошенный ресурс не найден
409	Конфликт состояния ресурса, например занятый столик или повторный отзыв
422	Ошибка валидации входных данных
500	Внутренняя ошибка сервера

Единый формат ошибки содержит поле code с машинно-читаемым кодом и поле message с текстовым описанием. Для ошибок валидации дополнительно возвращается список fields с описанием ошибок по конкретным полям.

Особенности проектирования

- Кухни вынесены в отдельную сущность RestaurantCuisine, так как один ресторан может относиться к нескольким кухням.
- Столики вынесены в RestaurantTable, чтобы бронирование ссылалось на конкретный столик и учитывало его вместимость.
- Reservation содержит RestaurantID и TableID. Это упрощает API и выборки бронирований по ресторану, но при реализации нужно проверять, что столик принадлежит выбранному ресторану.
- Review связан с ReservationID, чтобы пользователь мог оставить отзыв только после факта бронирования.
- Фотографии хранятся во внешнем источнике, а в API и БД передается путь или URL через filePath.
- Поле rating у ресторана рассматривается как агрегированное значение на основе отзывов, а avgBill используется для фильтрации по среднему чеку.

Вывод

В ходе работы был спроектирован REST API для сервиса бронирования столиков в ресторанах. Были описаны функциональные и нефункциональные требования, выбран формат REST API, подготовлена OpenAPI-схема с эндпоинтами, телами запросов и ответов, а также типовыми ошибками. Полученная схема может быть открыта в Swagger UI и использована как основа для дальнейшей backend-реализации приложения.